3 · Exp 3 — Insert into an array (beginning, any position, end)

The first "data structure" experiment. The skill here, shifting elements in a loop, comes back in Exp 4 and inside insertion sort (Exp 7).

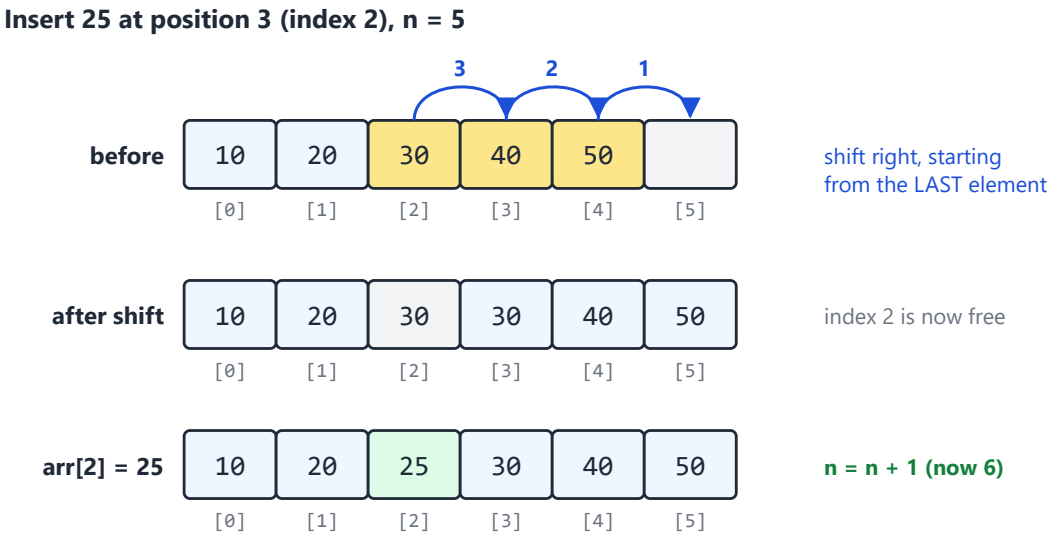
Array = fixed boxes → **Make a gap by shifting right** → Drop the value in → n = n + 1

3.1 The idea

An array's elements sit side by side and can't move by themselves. To put a value at index 2 you must first **make a gap** there by shifting everything from index 2 onward one place to the right.

GUESS FIRST

To shift `10 20 30 40 50` right from index 2, should the loop copy **30 first** or **50 first**? Try both on paper and see what goes wrong.



Insert 25 at position 3. The arrows are numbered in the order the loop does them.

If you copy 30 first, `arr[3] = arr[2]` overwrites 40 before 40 has moved: you get `10 20 30 30 30 30`. So the loop must run **from the end down**: `for (i = n; i >= pos; i--) arr[i] = arr[i-1];`.

POSITION VS INDEX

The program asks for a **position** counted from 1 (what humans say), but C++ indexes from 0. Position 3 = `arr[2]`, hence `arr[pos - 1] = value;`. Valid positions for insertion are **1 to n+1** (n+1 = "after the last one").

3.2 The program (corrected)

The manual's logic was right. This version adds the two checks it was missing: free space (`n < 100`) and a valid position.

E03_ARRAY_INSERT.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, value, pos, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n >= 100)                           // FIX: no room left for one more
    {
        cout << "Array is full. Insertion not possible.";
        return 0;
    }

    cout << "\n1. Insert at Beginning";
    cout << "\n2. Insert at Any Position";
    cout << "\n3. Insert at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Enter value: ";
        cin >> value;

        for (int i = n; i > 0; i--)         // shift right, last element first
            arr[i] = arr[i - 1];

        arr[0] = value;
        n++;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        if (pos < 1 || pos > n + 1)         // FIX: positions 1..n+1 are valid
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Enter Value: ";
        cin >> value;

        for (int i = n; i >= pos; i--)      // shift right from the end down to pos
            arr[i] = arr[i - 1];

        arr[pos - 1] = value;               // position 3 = index 2
        n++;
    }
    else if (choice == 3)
    {
```

```
        cout << "Enter Value: ";
        cin >> value;

        arr[n] = value;                    // first free slot is index n
        n++;
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

The manual's three sample inputs, run for real:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 1
Enter value: 5

Updated Array:
5 10 20 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 2
Enter Position: 3
Enter Value: 25

Updated Array:
10 20 25 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
```

```
Enter your choice: 3
Enter Value: 60

Updated Array:
10 20 30 40 50 60
```

The added checks working (position 7 in a 3-element array):

```
SCREEN (REAL RUN)
Enter number of elements: 3
Enter array elements:
1 2 3

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 2
Enter Position: 7
Invalid position.
```

Case	Loop	Why
Beginning	`for (i = n; i > 0; i--) arr[i] = arr[i-1];` then `arr[0] = value;`	every element moves
Position	`for (i = n; i >= pos; i--) arr[i] = arr[i-1];` then `arr[pos-1] = value;`	elements from index pos−1 onward move
End	`arr[n] = value;`	first free slot is index n; nothing moves

All three end with `n++`: the array now holds one more element.

WRITE IN YOUR RECORD

Aim: To insert an element into an array at the beginning, at a given position, and at the end.

Algorithm: read n and the elements → read choice → *beginning:* shift all right (from the end), `arr[0] = value` → *position:* check 1 ≤ pos ≤ n+1, shift right from the end down to index pos−1, `arr[pos-1] = value` → *end:* `arr[n] = value` → `n++` → print.

Complexity: beginning O(n), position O(n) (worst case), end O(1). Space O(1) extra.

Result: 10 20 30 40 50 became 5 10 20 30 40 50 (beginning), 10 20 25 30 40 50 (25 at position 3), 10 20 30 40 50 60 (end).

MISTAKES THAT COST MARKS

- Forgetting `n++`: the new last element never prints.
- Shifting left-to-right: duplicates the value (see the guess above).
- `arr[pos] = value` instead of `arr[pos-1]`: everything lands one place too far right.

3.3 Viva questions

Question	Answer
Why is insertion at the beginning O(n)?	All n elements must shift one place right.
Why is insertion at the end O(1)?	No shifting: write at index n (if there is space).
What is the drawback of arrays for insertion?	Fixed size, and shifting costs O(n). A linked list inserts by changing pointers.
Why `int arr[100]` and not `int arr[n]`?	Array sizes must be constants in standard C++; 100 is the maximum capacity, n is how many are in use.
What if n is already 100?	Overflow: there's no room. The program must refuse (our first check).

3.4 If the examiner changes the question

The manual's ASSIGNMENT list. Each function is tested (10 checks, all pass) and is written to drop into the program above. Each returns the new n.

Insert at a position (reused by the next two):

```
int insertAt(int arr[], int n, int pos, int value)   // pos is 1-based
{
    if (n >= CAP || pos < 1 || pos > n + 1)
        return n;                                    // no space / bad position
    for (int i = n; i >= pos; i--)
        arr[i] = arr[i - 1];
    arr[pos - 1] = value;
    return n + 1;
}
```

Insert multiple elements:

```
// Insert k values (vals[0..k-1]) at the end, stopping if the array fills up
int insertMultiple(int arr[], int n, int vals[], int k)
{
    for (int i = 0; i < k; i++)
    {
        if (n >= CAP)
        {
            cout << "Array full, " << k - i << " value(s) not inserted" << endl;
            break;
        }
        arr[n] = vals[i];
        n++;
    }
    return n;
}
```

Insert into a sorted array, keeping it sorted. This is the core move of insertion sort (Exp 7):

```
// arr is sorted ascending; keep it sorted
int insertSorted(int arr[], int n, int value)
{
    if (n >= CAP)
        return n;
    int i = n - 1;
    while (i >= 0 && arr[i] > value)     // shift bigger ones right (like insertion sort)
    {
        arr[i + 1] = arr[i];
        i--;
    }
    arr[i + 1] = value;
    return n + 1;
}
```

Insert only if space is available:

```
int insertIfSpace(int arr[], int n, int value)
{
    if (n >= CAP)
    {
        cout << "No space available. Insertion not possible." << endl;
        return n;
    }
    arr[n] = value;                      // space exists: put it at the end
    return n + 1;
}
```

Insert before / after a given element: search for the element, then reuse `insertAt`.

```
// Insert value just before the first occurrence of target
int insertBefore(int arr[], int n, int target, int value)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == target)
            return insertAt(arr, n, i + 1, value);   // index i = position i+1
    cout << target << " not found" << endl;
    return n;
}
```

```
int insertAfter(int arr[], int n, int target, int value)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == target)
            return insertAt(arr, n, i + 2, value);   // one place further right
    cout << target << " not found" << endl;
    return n;
}
```

In `main` you call these like `n = insertSorted(arr, n, value);`.

> **CHECK 3.1**
> `arr = 4 8 15`, n = 3. Trace the "any position" branch for pos = 2, value = 6: which assignments run, in order, and what is the final array?

CHECK 3.2

Is `pos = 4` valid for this n = 3 array? Is `pos = 5` ?

Answers

Check 3.1: i = 3: `arr[3] = arr[2]` (15); i = 2: `arr[2] = arr[1]` (8); the loop stops (i = 1 < pos). Then `arr[1] = 6`, n = 4 → **4 6 8 15**.

Check 3.2: pos = 4 = n+1 is valid (insert at the end). pos = 5 is invalid: it would leave a gap at index 3.